

Week 1: OWASP Top 10 Vulnerabilities Research Report

**Vortex Tech Cyber Security Internship Track Author: Ibraheem
Ahmed**

Introduction

The OWASP (Open Web Application Security Project) Top 10 is the industry-standard reference for the most critical web application security risks. It's updated periodically based on real breach data, vulnerability scan results, and input from thousands of security professionals worldwide. The most recent version is the **OWASP Top 10:2025**, which replaced the long-standing 2021 edition.

This report covers five vulnerabilities from the current list: what they are, how attackers exploit them, a real-world example of each, and how developers can prevent them.

1. Broken Access Control (A01)

What it is

Broken Access Control happens when an application fails to properly enforce what a logged-in user is and isn't allowed to do. Think of it like a hotel where your key card is supposed to only open your room but because of a mistake in how the locks were programmed, your card opens every room on your floor. The user isn't "breaking in" through a technical trick; the system just never checked whether they should be let in.

How an attacker exploits it

A common technique is called **ID manipulation** or **IDOR (Insecure Direct Object Reference)**. If a website shows your invoice at a URL like `mystore.com/invoice?id=1023`, an attacker might simply change the number to `id=1024` and see if it loads someone else's invoice. If the server never checks "does this invoice actually belong to the person asking for it?", the attacker gets free access to other people's data just by guessing or incrementing numbers.

Real-world example

This category is consistently ranked the **#1** risk because it appears in such a huge share of tested applications and because it directly leads to data exposure. Numerous large-scale data leaks over the years have traced back to APIs and web apps that let one user pull another user's account records simply by changing an ID number in a request no password cracking or malware required.

Prevention

- Enforce authorization checks on the **server side**, every single time a resource is requested never trust the client (browser/app) to only ask for its own data.

- Use unpredictable, non-sequential identifiers (like UUIDs) instead of simple incrementing numbers where feasible.
- Default to "deny access" and explicitly grant permissions, rather than assuming access unless blocked.

2. Security Misconfiguration (A02)

What it is

This vulnerability isn't a flaw in code it's a mistake in how a system is *set up*. Examples include leaving default admin passwords unchanged, having unnecessary features or ports enabled, exposing detailed error messages to the public, or forgetting to apply a security patch. It's the digital equivalent of installing a great lock on your front door and then leaving it unlocked because you forgot to turn the key.

How an attacker exploits it

Attackers routinely scan the internet for systems still using default credentials (like admin/admin) or cloud storage buckets that were accidentally left publicly accessible. No custom hacking tool is needed they just try well-known defaults or use automated scanners that check thousands of servers for open, misconfigured settings.

Real-world example

Misconfigured cloud storage has caused some of the most damaging breaches in recent memory. Companies have accidentally left cloud storage buckets set to "public" instead of "private," exposing millions of customer records names, addresses, even financial details to anyone who happened to find the link, without any password required at all.

Prevention

- Change all default credentials immediately after installation.
- Turn off unused features, ports, services, and sample applications that ship with a platform by default.
- Automate configuration checks (using tools that scan for open storage buckets, exposed admin panels, etc.) as part of the deployment pipeline rather than relying on manual review.

3. Software Supply Chain Failures (A03)

What it is

Modern applications are rarely built entirely from scratch — they rely on dozens or hundreds of third-party libraries, open-source packages, and tools. This vulnerability category covers what happens when something in that chain is compromised: a malicious package gets published, a legitimate library is hijacked, or a build system is tampered with — and that bad code rides along into your application without you writing a single malicious line yourself.

How an attacker exploits it

One common method is **dependency confusion or typosquatting**: an attacker publishes a malicious package with a name nearly identical to a popular one (e.g., `requests` instead of `requests`), hoping a developer mistypes it or that a company's internal build system accidentally pulls the public (malicious) version instead of their private one. Once installed, the malicious code runs with the same trust and permissions as the rest of the application.

Real-world example

The 2017 Equifax breach is one of the most cited examples in this space attackers exploited a known, unpatched vulnerability in an open-source web framework the company relied on (Apache Struts), exposing sensitive data for roughly 147 million people. Even though a patch existed, it hadn't been applied in time, showing how a single weak link in the software supply chain can have massive consequences.

Prevention

- Keep an up-to-date inventory of all third-party dependencies (a "Software Bill of Materials") so you know exactly what's running in production.
- Apply security patches promptly don't let known vulnerabilities sit unpatched.
- Use automated dependency-scanning tools that flag packages with known vulnerabilities before they're deployed.

4. Cryptographic Failures (A04)

What it is

This covers situations where sensitive data — passwords, credit card numbers, health records, personal information isn't properly protected through encryption, either while it's stored ("at rest") or while it's being transmitted ("in transit"). It also includes using outdated or weak encryption algorithms that can crack with modern computing power.

How an attacker exploits it

If a website transmits login credentials over plain, unencrypted HTTP instead of HTTPS, an attacker on the same network (like public Wi-Fi) can intercept that traffic and read the password directly this is called a "man-in-the-middle" attack. Similarly, if a database stores passwords in plain text instead of being hashed, a single database breach hands the attacker every user's actual password instantly.

Real-world example

Several major breaches over the years have been made significantly worse because passwords were stored in weakly hashed or even plain-text form. When attackers gained access to the underlying database, they didn't need to "crack" much of anything, the sensitive data was essentially handed to them because proper cryptographic protection wasn't in place.

Prevention

- Encrypt sensitive data both at rest (in the database) and in transit (using HTTPS/TLS everywhere).
- Never store passwords in plain text use strong, modern hashing algorithms designed for passwords (like bcrypt or Argon2), which are intentionally slow to resist brute-force cracking.
- Regularly review and retire outdated cryptographic algorithms as they become insecure over time.

5. Injection (A05)

What it is

Injection happens when an application takes untrusted input from a user and mixes it directly into a command it sends to another system like a database without properly separating "data" from "instructions." The most well-known form is **SQL Injection**, where an attacker slips database commands into a form field instead of the expected data (like a username).

How an attacker exploits it

Imagine a login form that builds a database query by directly pasting in whatever the user types. If a developer writes code that blindly inserts user input into a query string, an attacker can type something like **'OR '1'='1** into the username field. Instead of being treated as a name to search for, that text gets interpreted as part of the actual database command, tricking it into returning data it shouldn't potentially logging the attacker in as any user, or dumping an entire table of records.

Real-world example

SQL Injection has been behind some of the internet's most significant breaches including large-scale credit card theft operations where attackers used injection flaws in retail and payment systems to siphon off millions of records over time, often going undetected for months. It remains one of the oldest and most well-understood vulnerabilities, yet it still shows up regularly because input isn't always properly separated from commands.

Prevention

- Use **parameterized queries** (also called prepared statements), which treat user input strictly as data and never as executable code this is the single most effective fix for SQL injection.
- Validate and sanitize all user input on the server side.
- Apply the principle of least privilege to database accounts, so even if an injection succeeds, the damage it can do is limited.

Personal Reflection:

Of the above-mentioned 5 vulnerabilities, Security Misconfiguration was the most surprising to me. I expected that the most dangerous vulnerabilities require sophisticated hacking skills, but attack is too simple to apply yet too dangerous, a default password never changed, or a storage setting left public by mistake. It's surprising that some of the largest data breach happened not because of skilled and brilliant attackers but because of a checkbook that nobody remembers to check. It reminds me that security is not just about technical skills but also about discipline and processes.